

Assignment 4 (WPT)

Exercise 1: Create a function processData that takes two parameters: a string and a callback function. Your task is to write a callback that converts the string to uppercase and then call it within processData.

Requirements: • Define a function toUpperCase that will serve as a callback.

• Pass a string and toUpperCase to processData and log the output.

```
<!DOCTYPE html>
```

```
<body>
```

```
  <script>
```

```
    // Callback function to convert string to uppercase
```

```
      function toUpperCase(str) {
```

```
        return str.toUpperCase();
```

```
    }
```

```
    function processData(input, callback) {
```

```
      const result = callback(input);
```

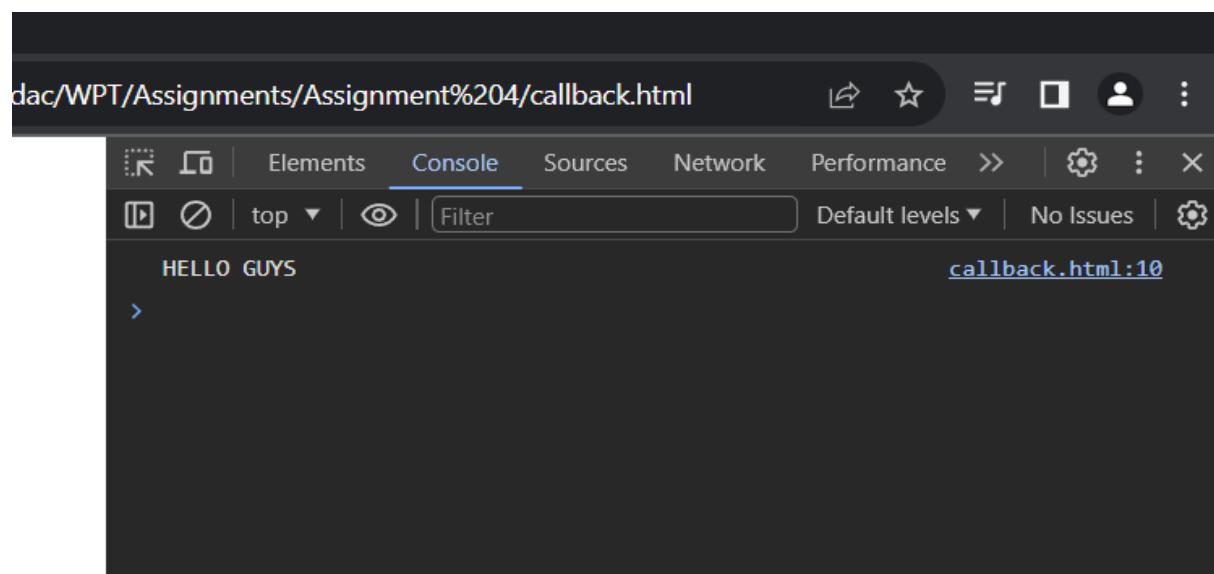
```
      console.log(result);
```

```
    }
```

```
    processData("hello guys", toUpperCase);
```

```
  </script>
```

```
</body>
```



Exercise 2: Write a function `forEachElement` that accepts an array and a callback. This function should apply the callback to each element of the array.

Requirements: ● Pass an anonymous function as the callback that multiplies each element by 2 and logs the result with the index.

```
<!DOCTYPE html>

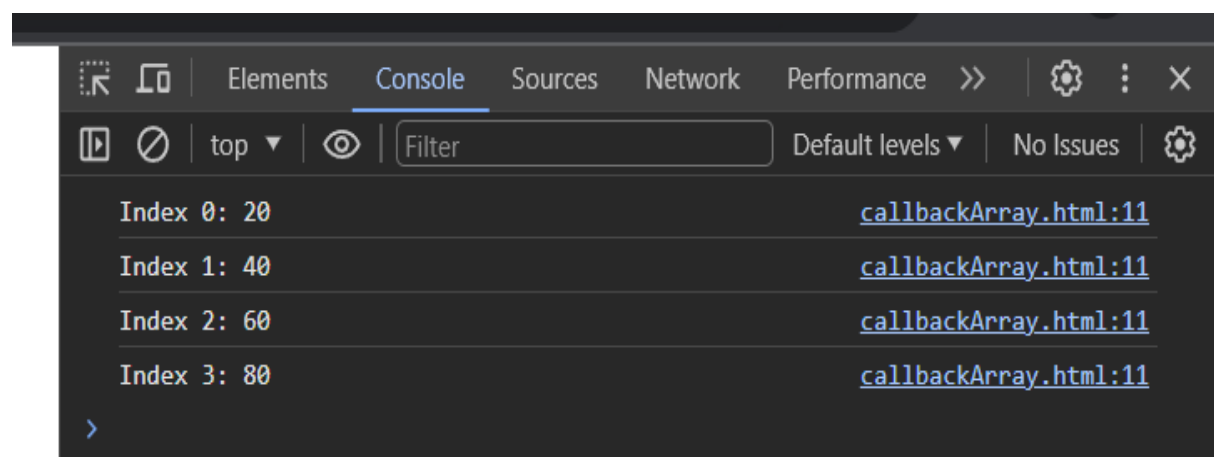
<body>

  <script>

    function forEachElement(arr, callback) {
  for (let i = 0; i < arr.length; i++) {
    callback(arr[i], i);
  }
}

forEachElement([10, 20, 30, 40], function(element, index) {
  console.log("Index " + index + ": " + (element * 2));
});

  </script>
</body>
```



Exercise 3: Simulate a network request by creating a function `fetchData` that takes a URL and a callback as parameters. Use `setTimeout` to simulate a delay and then call the callback with a string representing a response.

Requirements: ● After a delay, log the “response” to the console.

```
<!DOCTYPE html>

<body>

  <script>

    function fetchData(url, callback) {

      console.log("Fetching data from " + url);

      setTimeout(() => {

        callback("Simulated response data");

      }, 1000);

    }

    fetchData("https://www.google.com/", function(response) {

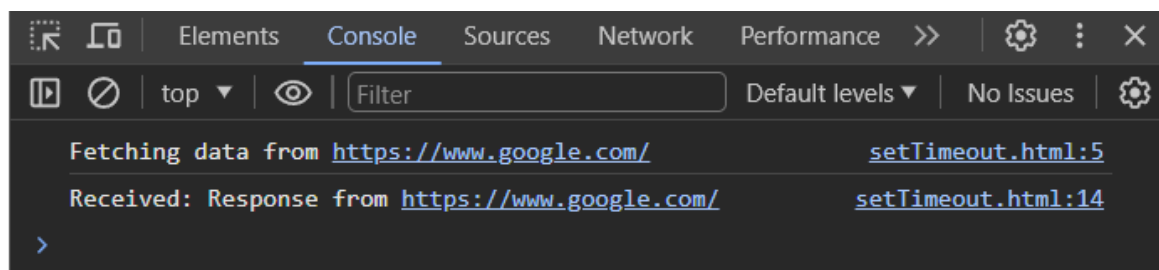
      console.log("Received:", response);

    });

  </script>

</body>
```

Received Response after 2 seconds.



Exercise 4: Modify fetchData from Exercise 3 to include error handling.
Requirements: ● Call the callback with an error message if an error occurs; otherwise, pass the “response.”

- Handle the error gracefully by logging it if it occurs.

```
<!DOCTYPE html>
```

```
<body>
```

```
  <script>
```

```
function fetchData(url, callback) {
```

```
  console.log("Fetching data from " + url);
```

```
  setTimeout(() => {
```

```
    if (url.includes("error")) {
```

```
      callback("Error: Failed to fetch data", null);
```

```
    } else {
```

```
      callback(null, "Simulated response data");
```

```
    }
```

```
  }, 1000);
```

```
}
```

```
fetchData("https://www.google.com", (error, response) => {
```

```
  if (error) {
```

```
    console.error("Error:", error);
```

```
  } else {
```

```
    console.log("Response:", response);
```

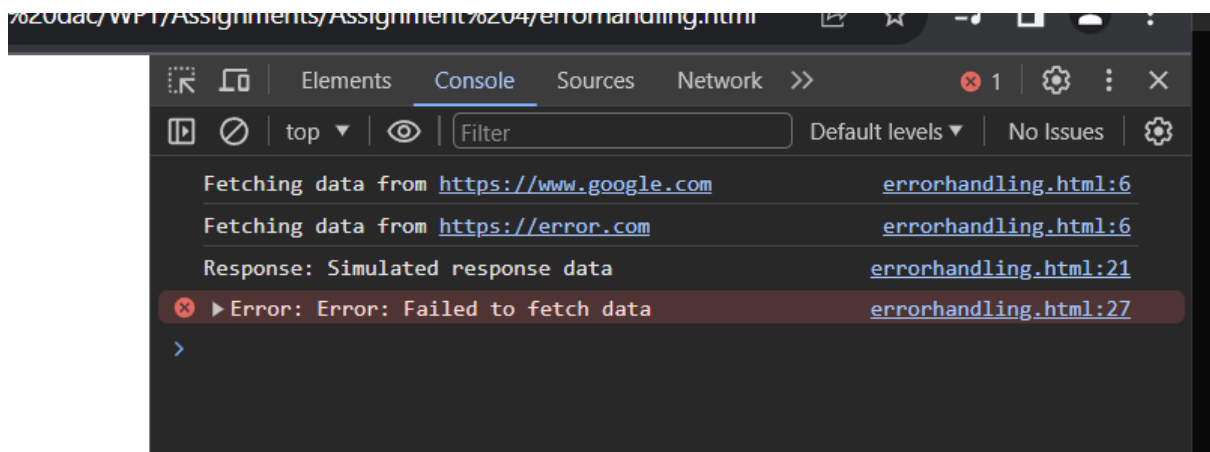
```
  }
```

```
});
```

```
fetchData("https://error.com", (error, response) => {
```

```
  if (error) {
```

```
    console.error("Error:", error);
  } else {
    console.log("Response:", response);
  }
});
</script> </body>
```



Exercise 5: Using `fetchData` from Exercise 4, create another function `processData` that simulates processing the fetched data. Chain these functions together using nested callbacks.

Requirements: ● First, call `fetchData`. Once the response is received, pass it to `processData`.

- `processData` should modify the data and log the processed result.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<script>
```

```
function fetchData(url, callback) {  
    console.log("Fetching data from " + url);  
  
    setTimeout(() => {  
        if (url.includes("error")) {  
            callback("Error: Failed to fetch data", null);  
        } else {  
            callback(null, "Simulated response data");  
        }  
    }, 1000);  
}
```

```
function processData(data, callback) {  
    console.log("Processing data...");  
    setTimeout(() => {  
        const processedData = data;  
        callback(processedData);  
    }, 500);  
}
```

```
fetchData("https://www.google.com", (error, response) => {  
  if (error) {  
    console.error("Fetch Error:", error);  
  } else {  
    processData(response, (processed) => {  
      console.log("Processed Data:", processed);  
    });  
  }  
});  
</script>  
</body>  
</html>
```

